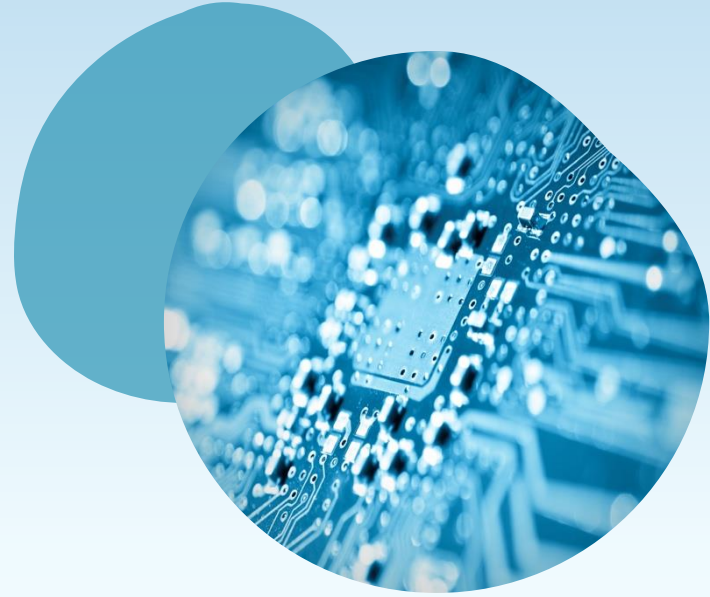


Software Building Blocks

Embedded Software for the Internet Of Things

Kasim Sinan Yildirim



Components for Embedded Programs

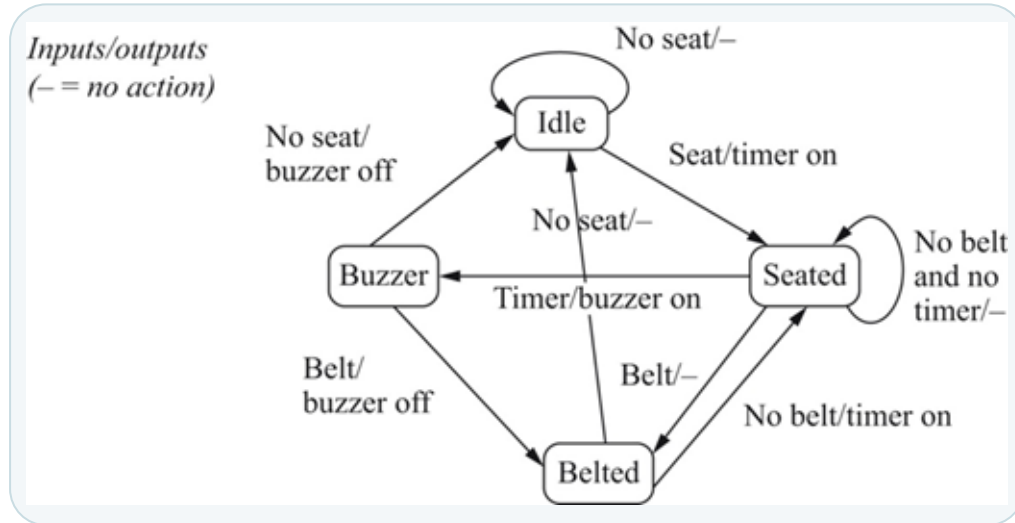
- Embedded Software is different and more challenging than writing typical PC programs:
 - Must provide rich functionality
 - Often run at a required rate to meet system deadlines
 - Fit into the allowed amount of memory
 - Meet power consumption requirements
- We have some **basic building blocks** for our programs
 - State machines, circular buffers, queues,...

Finite State Machines (FSMs)

- Embedded systems react to their inputs.
- The reaction can be characterized in terms of the input received and the current state of the system
 - Looks like a **finite-state machine**!
- Implement the program in a state machine style (as in the hardware design)

Finite State Machines (FSMs)

- State machine example: simple seat belt controller.



Finite State Machines (FSMs)

- State machine example: simple seat belt controller.

```
#define IDLE 0
#define SEATED 1
#define BELTED 2
#define BUZZER 3

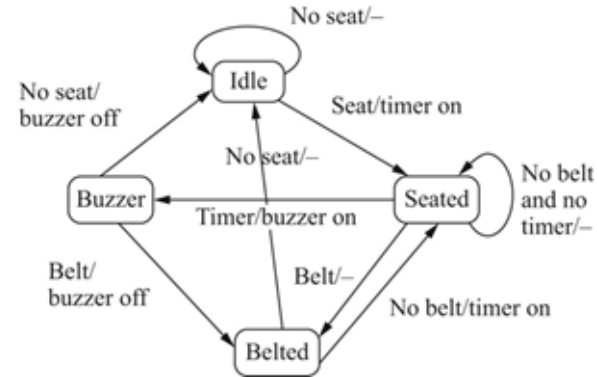
switch(state) { /* check the current state */
case IDLE:
    if (seat){ state = SEATED; timer_on = TRUE; }
    /* default case is self-loop */
    break;

case SEATED:
    if (belt) state = BELTED; /* won't hear the buzzer */
    else if (timer) state = BUZZER; /* didn't put on belt in time */
    /* default case is self-loop */
    break;

case BELTED:
    if (!seat) state = IDLE; /* person left */
    else if (!belt) state = SEATED; /* person still in seat */
    break;

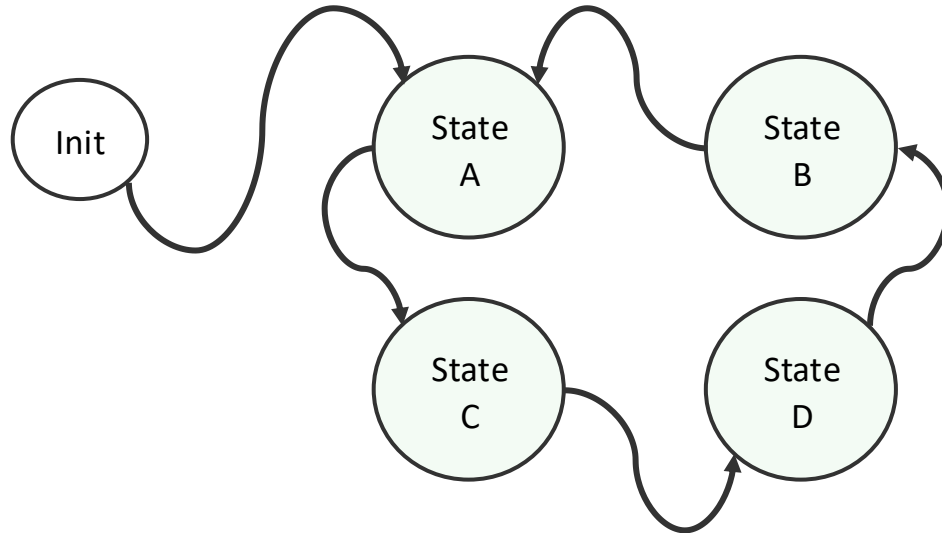
case BUZZER:
    if (belt) state = BELTED; /* belt is on---turn off buzzer */
    else if (!seat) state = IDLE; /* no one in seat---turn off buzzer */
    break;
}
```

*Inputs/outputs
(- = no action)*



Function Pointers & State Machines

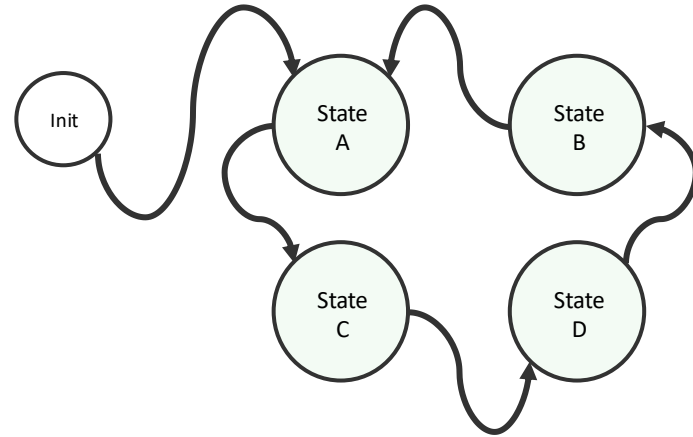
- Good idea to use the function pointer implementation if the state machine has a lot of states.
 - Decreases the complexity.



Function Pointers & State Machines

- Define states and state machine structure:

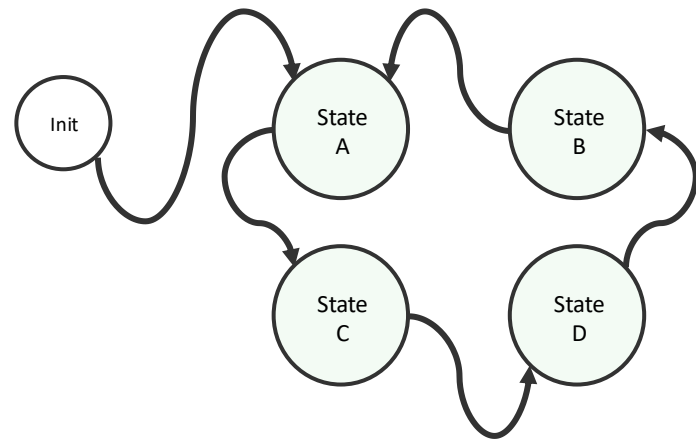
```
typedef enum{  
    STATE_A;  
    STATE_B;  
    STATE_C;  
    STATE_D;  
    NUM_STATES  
}State_t;  
  
typedef struct{  
    State_t state;    /* defines the command */  
    void (*func)(void); /* defines the function to execute */  
} StateMachine_t;
```



Function Pointers & State Machines

- Declare functions that implement states, variable to hold current state and the state machine itself:

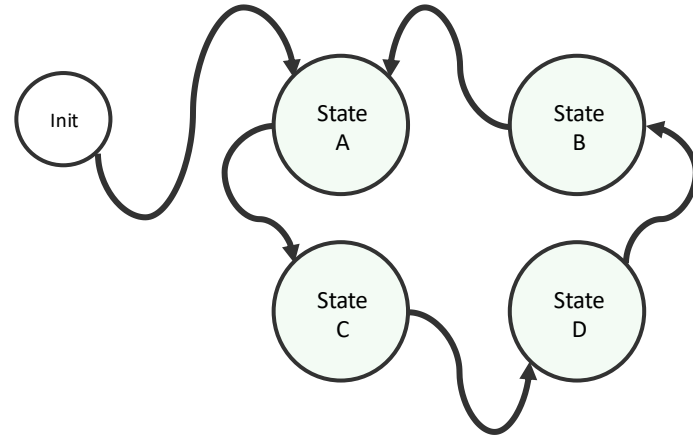
```
/* state machine function prototypes */  
void fn_StateA(void);  
void fn_StateB(void);  
void fn_StateC(void);  
void fn_StateD(void);  
  
/* variable that holds the current state */  
State_t cur_state = STATE_A;  
  
StateMachine_t StateMachine[] = {  
    {STATE_A, fn_StateA},  
    {STATE_B, fn_StateB},  
    {STATE_C, fn_StateC},  
    {STATE_D, fn_StateD}  
};
```



Function Pointers & State Machines

- Fill state functions:

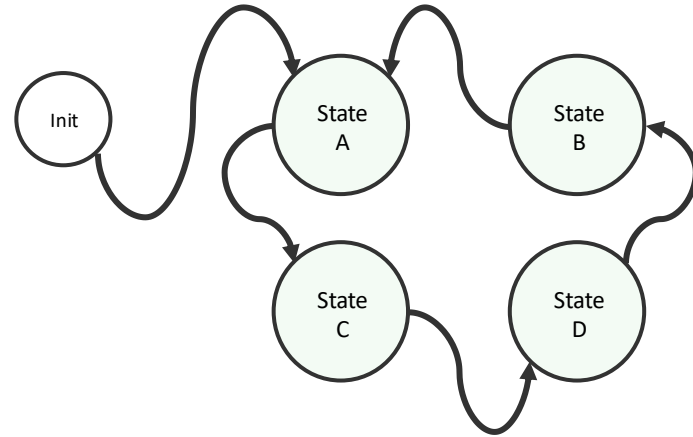
```
void fn_StateA(void){  
    /* add the necessary code here */  
  
    cur_state = STATE_B;  
}  
  
void fn_StateB(void){  
    /* add the necessary code here */  
  
    cur_state = STATE_C;  
}  
  
void fn_StateC(void){  
    /* add the necessary code here */  
  
    cur_state = STATE_D;  
}  
  
void fn_StateD(void){  
    /* add the necessary code here */  
  
    cur_state = STATE_A;  
}
```



Function Pointers & State Machines

- Very simple “run” function:
 - Called each time there is an event.

```
void run(void) {  
    if(cur_state < NUM_STATES){  
        (*StateMachine[cur_state].func) ();  
    }  
    else{  
        // error  
    }  
}
```



Circular Buffer

- Stream-oriented processing:
 - data comes in regularly and must be processed on the fly.
 - Use to store **sensor readings** (e.g., temperature, humidity, ...)
- Reuses fixed memory
 - New values overwrite old values

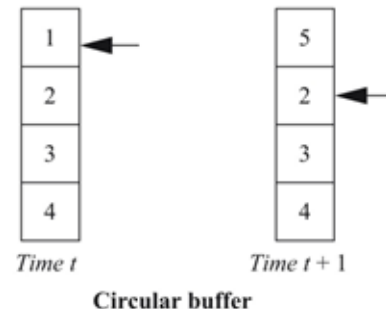
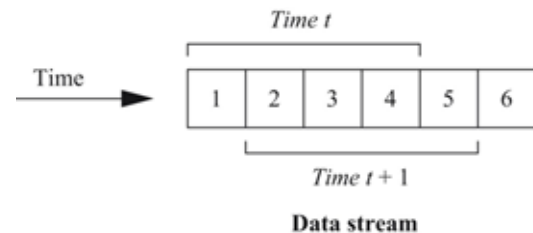


Circular Buffer in C

- Use an **array** as the buffer that has always has a **fixed** number of data
 - **pos**: the position of the current sample.
 - moves as new values are added to the buffer (wraparound)

```
#define CMAX 6 /* filter order */
int circ[CMAX]; /* circular buffer */
int pos; /* position of current sample */

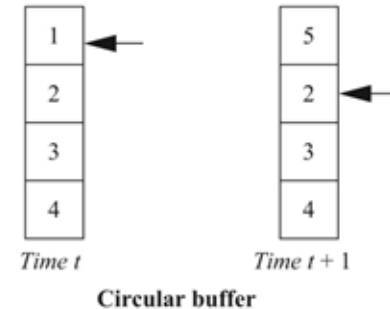
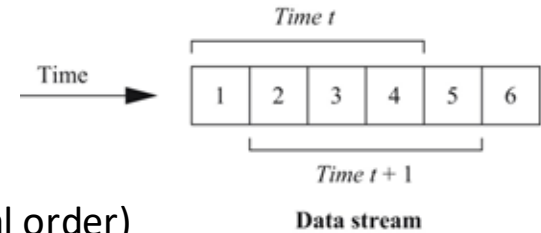
void circ_update(int xnew) {
    /* compute the new head value with wraparound;
    the pos pointer moves from 0 to CMAX-1 */
    pos = ((pos == CMAX-1) ? 0 : (pos+1));
    /* insert the new value at the new head */
    circ[pos] = xnew;
}
```



Circular Buffer in C

- Initialization:
 - set the buffer values and **pos** to zero.
- Get:
 - return the i th value of the buffer (translate the index in temporal order)

```
void circ_init() {  
    int i;  
    for (i=0; i<CMAX; i++) /* set values to 0 */  
        circ[i] = 0;  
    pos=CMAX-1; /* start at tail so first element will be at 0 */  
}  
  
int circ_get(int i) {  
    int ii;  
    /* compute the buffer position */  
    ii = (pos - i) % CMAX;  
    return circ[ii]; /* return the value */  
}
```



Queues

- We use queues for data that arrive and depart at unpredictable times.
 - Queues are often referred to as an **elastic buffer**.
- One way to build a queue is with a **linked list**
 - allows the queue to grow to an arbitrary size.
 - the overhead of dynamic memory allocation
- Another way: use an array to hold all the data.
 - The queue may have varying numbers of elements in it.
 - circular buffer always has a fixed number of data

Queues in C

```
#define Q_SIZE 8          /* your queue size may vary */
#define Q_MAX (Q_SIZE-1) /* this is the maximum index value into the array */
int q[Q_SIZE];           /* the array for our queue */
int head, tail;          /* indexes for the current queue head and tail */

void queue_init() {
    /* initialize the queue data structure */
    head = 0;
    tail = 0;
}
```

Queues in C

```
void enqueue(int val) {

    if (((tail+1) % Q_SIZE) == head) error("enqueue onto full queue",tail);
    q[tail] = val;
    /* update the tail */
    if (tail == Q_MAX)
        tail = 0;
    else
        tail++;
}

int dequeue() {
    int returnval;
    if (head == tail) error("dequeue from empty queue",head);
    returnval = q[head];

    if (head == Q_MAX)
        head = 0;
    else
        head++;

    return returnval;
}
```

Producer / Consumer

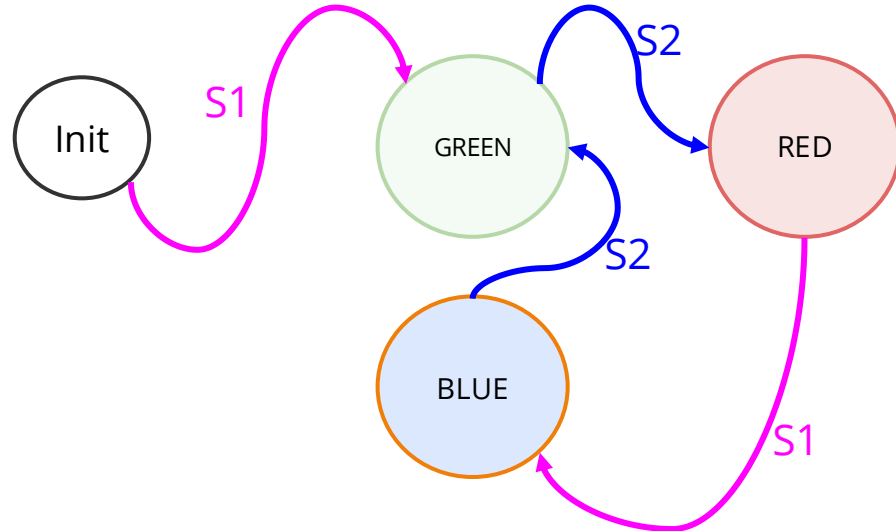
- Many systems may take in varying amounts of data over time and produce varying amounts.
 - the variable-rate output of one stage becomes the variable-rate input of another stage
 - We might need several buffers and queues!



Lab #1

- Implement the following FSM:

S1: switch S1 pressed
S2: switch S2 pressed



Buttons and LEDs

